of the Telegram bot token is shown in Fig. 4.

The only other data required is your chatid. To get it, find the bot named @GetIDsBot. You will be presented with a window describing your chat ID. Fig. 5 shows Telegram bot app ID.

Well, now you are all set with the credentials to run your chatbot in Telegram. Write down these two ghostly looking numbers clearly for using them in the sketch.

Python prerequisite

Two Python pip modules need to be installed to have the Python ready for Telegram: python-telegram-bot and telethon.

```
$ pip3 install python-telegram-bot
$ pip3 install telethon
```

In Linux, you may sudo it for installation, while on Windows just clicking would do. The script is used in Python3 version. Once you run the script file in the same directory where your @bera1bot.session file is residing, the data will start appearing endlessly.

```
somnath@somnath-Satellite-L855:~$ python3
get.py
```

Telegram with Python

The above setup is alright for getting messages on Telegram app on a mobile phone. But to get Telegram interfaced with Python, you must do the following:

In a browser open *https://my.telegram.org* and enter the same mobile number that you have set up on the mobile app (like +91942345xxxx). This will send you an SMS into the Telegram bot in the phone app of Telegram (Telegram channel inside Telegram). Enter that code into the next window. You will be ushered into anoher window where you have to give some name to the app title and short name (such as bera2 bera2).

Press Next and you will be presented with two important identities: api_id, api_hash. The

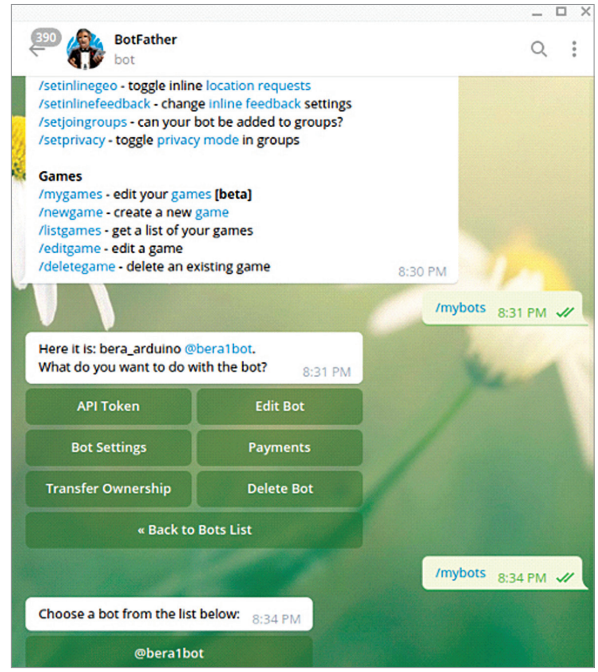

Fig. 3: Telegram bit setting

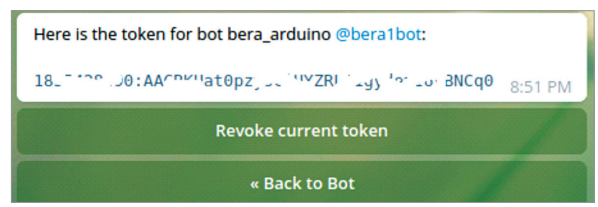

Fig. 4: Telegram bot token

username is already there, which you had setup while creating the bot @bera1bot. Now, only these three parameters are essential for accessing the incoming messages in a Telegram channel in a Python script. Let us see how it works.

With TelegramClient(name, api_id, api_hash) as client, run the code:

```
@client.on(events.NewMessage() )
async def handler(event):
  async for message in client.iter_
     messages('‘):
     print(message.text() )
 client.run_until_disconnected()
```

Well, that is the main loop for continuously getting messages as they arrive in the channel. You can get the above code with article and named revice.py and run on Python.

On first run of the Python code on computer, it will ask for the session parameters. For that, it will ask your phone number that you used earlier. On entering that, it will send a code through SMS into the Telegram bot in the phone app of Telegram. Enter that code in the next window and the session file will be created and your Python code will start working.

To make your code truly portable, you have to give away both the files—the Python code and the @bera1bot.session file together.

Arduino sender unit

The Arduino sketch is simple and straightforward. ESP32 is provided with a series of Wi-Fi usernames (IDs) and passwords. It connects with the suitable Wi-Fi connection first. If unsuccessful, it restarts on its own and selects the next ID/password. After establishing connection, it gets the data string and then using CHAT_ID and BOTtoken it creates a https datastring, which is pumped out to Telegram using http.GET(). An LED connected to pin 23 will blink continuously when the data uploading is taking place successfully.

After uploading the Arduino code, connect the ESP32 as shown in Fig. 6.

The sample project

The sample project is divided into two parts: the sender unit and the receiver unit. The sender unit is based on ESP32 with a DS18B20 connected to it. It creates a simple CSV data stream comprising date and time taken live from the NTP server. It adds up a few arbitrary data streams